V2X Message Confidence Attribution Services Based on Dead-Reckoning and Collaboration

Gustavo D'Angelo, João Gabriel Feres

I. Project Implementation

Both services are implemented in C++ as engines with no dependency on a simulator, each in its own repository with its own experiments. A wrapper repository gathers them as git submodules (Fig. 1); the experiments are standalone and the wrapper only pins their versions.

The two engines share `v2x-misc`, a header-only library of motion primitives free of any detection policy: WGS-84 geodesy, coordinate policies, the CTRA integrator, the OLS regression of the Service I predictor, the consistency check and a local tangent plane. Table I maps each element of the design to the code that implements it.

TABLE I
DESIGN ELEMENTS AND THE CODE THAT IMPLEMENTS THEM.

Design element	**Service I**	**Service II**
Message to motion vector	`MotionVectorAdapter`	`MessageAdapter`
Plausibility check	not an engine stage*	`plausibilityCheck`
Consistency check	`consistencyCheck`	`consistencyCheck` (`v2x-misc`)
Reference trajectory	`Predictor`, OLS-CTRA model	`ObjectTrack`: UKF
Predictive / gate check	`predictiveCheck`, $d_{\max}$	`Tracker::residual`, `gateCheck`
Association	—	`Tracker::associate`, `HungarianAlgorithm`
Fusion	—	`Tracker::kalmanUpdate`
Confidence	`BetaReputationEvaluator`	`Scorer`
Expiry	`Predictor::purge` (window)	`Tracker::retireStaleTracks`
Per-stage latency	`StageTiming`	`StageTimer`

*The VeReMi Extension traces carry only finite fields, so the runner feeds the engine without this screen.

A. Service I: *BeaconConfidenceEngine*

The engine composes a `Predictor` and a `Scorer` behind one call, `onMotionVector(senderId, mv)`, which returns the sender's confidence, the prediction error, the threshold and both verdicts (Fig. 2). The `Predictor` holds a bounded window per sender and the motion model; the `Scorer` holds one scalar per sender and delegates the update to an `IConfidenceEvaluator`. Three evaluators exist, the counterless Beta filter, an EWMA and a majority vote, so the scorer comparison of Section II-A replaces one component under identical upstream outcomes. Every constant lives in `MechanismConstants.h`, every tunable in `EngineConfig`, which also carries the ablation switches.

Two hosts drive the same engine: `veremi_runner` replays VeReMi Extension Parquet files for the detection experiments, and `offline_evaluator` replays dependency-free binary traces next to a decoupled F2MD reference [1] for the cost experiments, natively or on the RISC-V board.

B. Service II: *ObjectConfidenceEngine*

The engine is called once per received message with all its vectors, `onMotionVectors(vectors, receiver, now)`, and returns one `VectorRecord` per vector: outcome (dropped, started, fused, rejected), track, residual and confidence (Fig. 3). `MessageAdapter` builds the vectors: it projects positions on a local tangent plane and moves the CAM reference point to the bounding-box centre by the rigid-body transport of position and velocity. A `Tracker` owns the tracks and the source bindings; a `Scorer` owns the Beta evidence per track.

Two libraries are used unmodified, in addition to `Eigen`, the library for linear algebra. The UKF is at `libs/ukf/`; the `Tracker` calls `ProcessMeasurement` to initialise and update a track and `Prediction` on a copy of the filter to project it, converting the heading to the filter's yaw and the time to microseconds. The Hungarian solver is the implementation of Cong Ma (`hungarian-algorithm-cpp`); infeasible pairs enter it at a large finite cost so that feasible matches are preferred, and the gate is applied again after the solve.

`tools/replay` feeds the engine from a flat reception file, writes the records, and with `--profile` reports the latency of each stage through `StageTimer`. The same binary is cross-compiled statically for the RISC-V board.

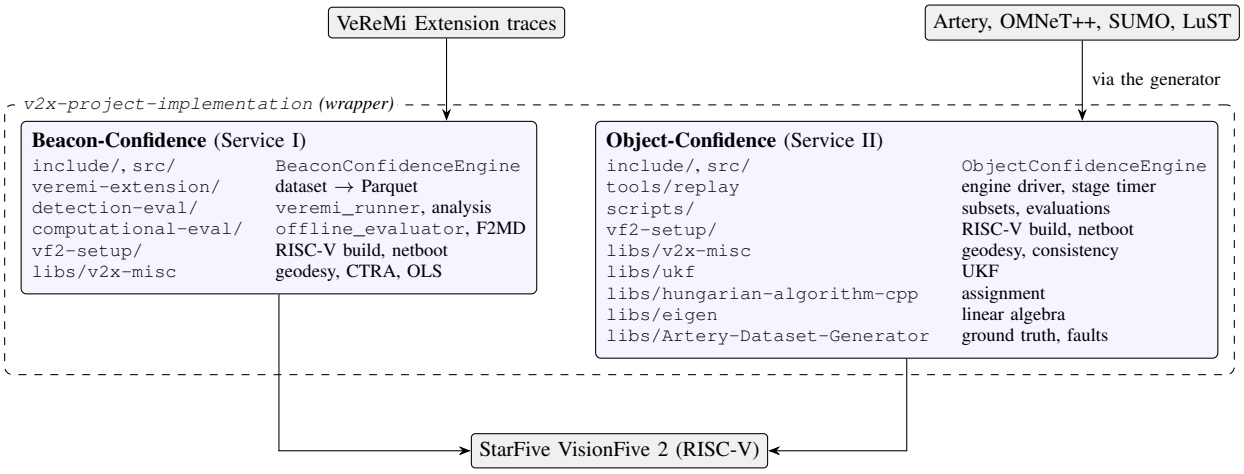


Fig. 1. The wrapper repository. Each service is a standalone repository with its engine, its experiments and its libraries as submodules. Service I is evaluated on VeReMi Extension; Service II on traces produced by the Artery dataset generator. Both cross-compile their engine for the RISC-V board.

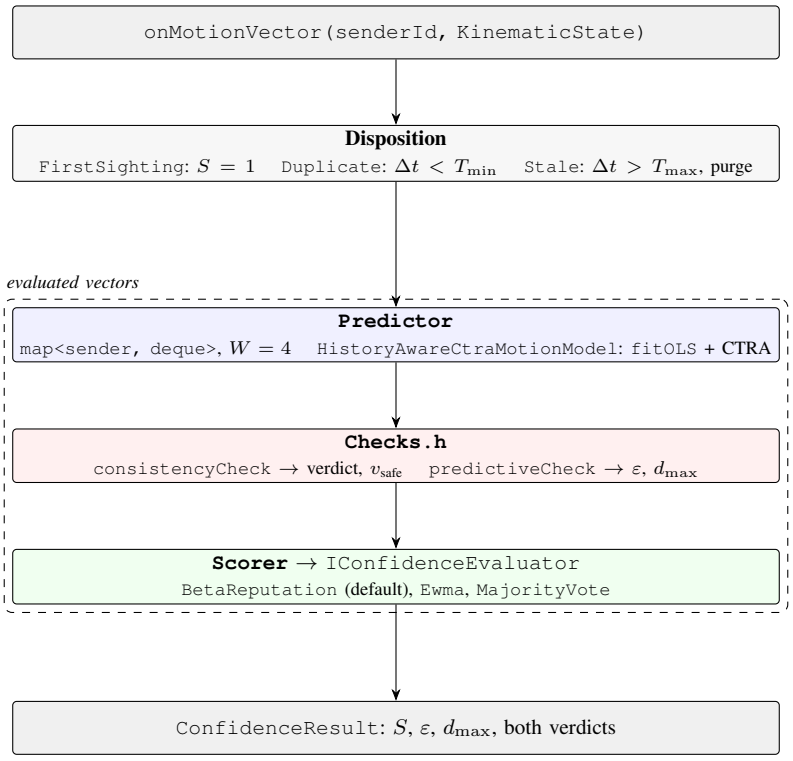


Fig. 2. `BeaconConfidenceEngine`, the implementation of the Service I pipeline of the design. Only evaluated vectors reach the predictor; the vector is appended to the sender's window after the update. `StageTiming` measures the predictor and the scorer.

C. Dataset generation for Service II

VeReMi Extension carries no CPM, so Service II needs its own data. The Artery Dataset Generator runs Artery [2] on the LuST scenario [3] and records honest ground truth; every error is added offline (Fig. 4). Table II gives the simulation parameters.

Ground truth. A `StationRecorder` per vehicle decodes every CAM and CPM as sent, and a `DatasetSink` writes JSON Lines tables: messages sent, receptions, the pose of every vehicle every 0.1 s, and vehicle metadata. Perceived objects carry two oracle fields, the station the object really is and the radars that track it, which the association metrics use and the engine never reads. The recorder also corrects Artery's deviations from the standards: headings rotated from SUMO grid north to true north, the CPM y axis pointing north, the signs of `measurementDeltaTime` and `velocityDirection`, and the CAM acceleration, yaw rate and dimensions taken from the vehicle data provider.

Faults. `inject_faults.py` applies the eight VeReMi Extension malfunctions with F2MD magnitudes [4], [1] to 30 % of the vehicles from $t = 10$ s, one type per subset, and can add the VeReMi sensor error models in future work. Position

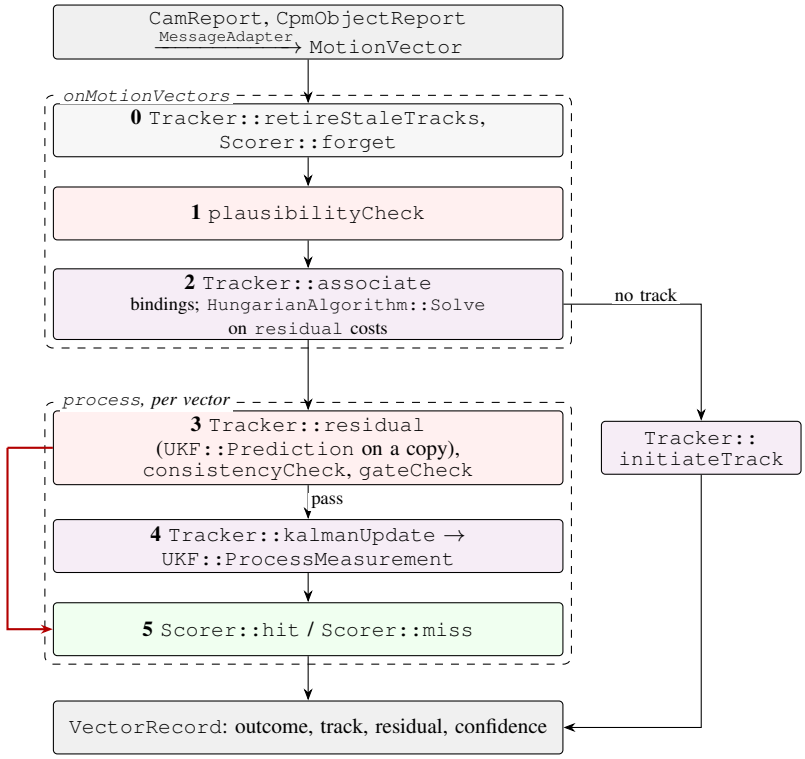


Fig. 3. `ObjectConfidenceEngine`, the implementation of the Service II pipeline of the design. The numbers are the stages `StageTimer` measures; `initiateTrack` is outside them.

TABLE II
SIMULATION PARAMETERS OF THE `LUST-FULL` RUN.

Parameter	Value	Source
Scenario	complete LuST, 40 s from 07:00	[3]
TX power / sensitivity	20 mW / −89 dBm	F2MD LuST configuration [4]
Propagation	GEMV^2	[5]
CAM generation	EN 302 637-2 triggers, DCC	[6]
CPM generation	TS 103 324 V2.1.1, DCC, penetration 1.0	[7]
Radars	65 m ±40° and 150 m ±5°, front and rear	[8], Table 4

malfunctions move the CAM position and the CPM reference position, and with it every perceived object; speed malfunctions reach the CAM only, since the CPM carries no speed of its author. A CAM and a CPM of the same instant share one GNSS fix. Every faulty message is labelled with its fault type.

Receiver view. `receiver_view.py` joins receptions and messages into one file per receiver, as the VeReMi trace files are organised, and filters nothing; each application filters its own input.

II. PRELIMINARY RESULTS

A. Service I

Service I was evaluated on VeReMi Extension [4], rush hour 07h–09h, one decision per received message, and benchmarked on a StarFive VisionFive 2 against F2MD re-executed under the same harness. Table III gives detection per fault. Seven of the eight faults reach F1 above 0.97; ConstPosOffset reaches 0.0192, the blind spot the design predicts: a constant offset cancels in every difference Service I computes.

The ablation (Table IV) supports the design of Service I: without the consistency check, speed faults and ConstPos go undetected; without the dynamic threshold, the false-positive rate rises from 1.83 % to 68.69 %; the OLS fit costs nothing measurable and is kept for payload trust independence, since it derives acceleration and yaw rate from the observed window rather than from the payload. The history window peaks at $W = 4$ (Fig. 5). Among the scorers (Table V), F1 is comparable; the Beta filter evicts within 1–2 messages and has the highest ROC AUC.

On the VisionFive 2 (Table VI), the engine takes 1.45 μs per message end to end, 75.9 to 94.0 times less than the F2MD combinations.

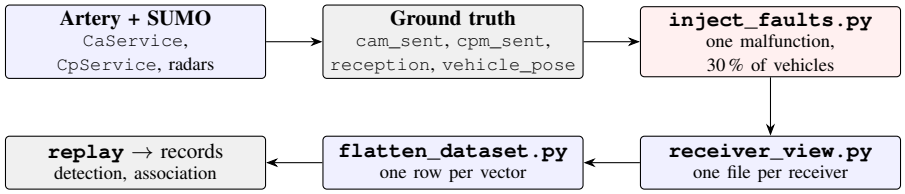


Fig. 4. From simulation to results. Everything the simulation writes is ground truth; faults and sensor errors are injected offline, so one run yields every subset.

TABLE III
SERVICE I DETECTION ON VEREMI EXTENSION, 07H–09H; BASELINE F1 AS PUBLISHED.

Fault	[4]	[9]	Service I			
	F1	F1	F1	Recall	Prec.	Acc.
ConstPos	0.9928	0.9896	0.9748	99.75%	95.31%	98.50%
ConstSpeed	0.8843	0.9704	0.9736	99.51%	95.29%	98.43%
ConstPosOffset	0.9397	0.6445	0.0192	0.98%	36.36%	70.96%
ConstSpeedOffset	0.5814	0.9946	0.9706	98.85%	95.34%	98.26%
RandomPos	0.9983	0.9970	0.9760	99.75%	95.53%	98.57%
RandomSpeed	0.9625	0.9813	0.9756	99.75%	95.45%	98.55%
RandomPosOffset	0.9765	0.9671	0.9740	99.75%	95.16%	98.45%
RandomSpeedOffset	0.7828	0.9954	0.9748	99.67%	95.38%	98.50%

B. Service II

Setup. The `lust-full` run (Table II) has 271 vehicles, 21 835 CAMs and 54 573 CPMs, 13 668 of them with perceived objects, and 352 066 receptions. Each of the eight subsets applies one malfunction, without sensor errors, and yields 310 398 vectors (105 042 from CAMs, 205 356 CPM objects). As for Service I, a vector is flagged when the confidence of its track after the vector falls below 0.5, and is positive when its content was falsified; a CPM object takes the label of its message.

Detection (Table VII). On CAMs, position faults reach F1 0.92–0.93, except ConstPosOffset at 0.15: the offset vehicle builds its own track beyond the gate, and a track fed by a single source agrees with itself. Speed faults reach 0.26–0.65. Their CAMs keep a correct position, so they land on the vehicle's true track, which honest CPM objects from its neighbours keep confident: in ConstSpeed, 84 % of the faulty CAMs are rejected by the consistency check, but only 59 % of those leave their track below 0.5. On CPM objects, the false-positive rate is $8\,\%$ against $1.9\,\%$ on CAMs; speed faults never reach a CPM, so those rows have no positives.

Association (Table VIII), against the oracle station of each vector. Tracks are almost pure, $99.55\,\%$ of the vectors sharing their track's majority station, but fragmented: an object spreads over 3.7 tracks per receiver on average, and its honest vectors change track 30 times per 100. Honest CPM objects are rejected $10\,\%$ of the time against $3\,\%$ for CAMs; the residual of honest vectors has a 95th percentile of about 9–10 m, above the gate. Fragmentation and these rejections are the main open issues.

Cost (Table IX). On the VisionFive 2, with the first 46 504 messages of each subset (the complete 40 s of 32 of the 222

TABLE IV
SERVICE I ABLATION, F1, VEREMI EXTENSION 07H–09H.

Fault	Full	w/o consistency	w/o dynamic $d_{\max}$	w/o OLS
ConstPos	0.9748	0.0033	0.5433	0.9764
ConstPosOffset	0.0192	0.0016	0.4097	0.0192
ConstSpeed	0.9736	0.4566	0.5414	0.9751
ConstSpeedOffset	0.9706	0.0857	0.5390	0.9722
RandomPos	0.9760	0.9815	0.5428	0.9775
RandomPosOffset	0.9740	0.9525	0.5425	0.9756
RandomSpeed	0.9756	0.7292	0.5433	0.9771
RandomSpeedOffset	0.9748	0.1434	0.5431	0.9763
Macro F1	**0.8548**	0.4192	0.5256	0.8562
Macro FPR	**1.83%**	0.60%	68.69%	1.70%

TABLE V
SERVICE I CONFIDENCE FILTERS ON IDENTICAL CHECK OUTCOMES, 07H–09H.

Filter	Eviction	Recall	Prec.	F1	FPR	ROC AUC
Beta (proposed)	**1–2 msg**	87.25%	87.98%	0.8548	1.83%	**0.9859**
EWMA ($\lambda = 0.90$)	7 msg	83.54%	85.11%	0.8429	**0.96%**	0.9830
Majority vote ($K = 5$)	3 msg	87.24%	87.74%	**0.8566**	1.59%	0.9778

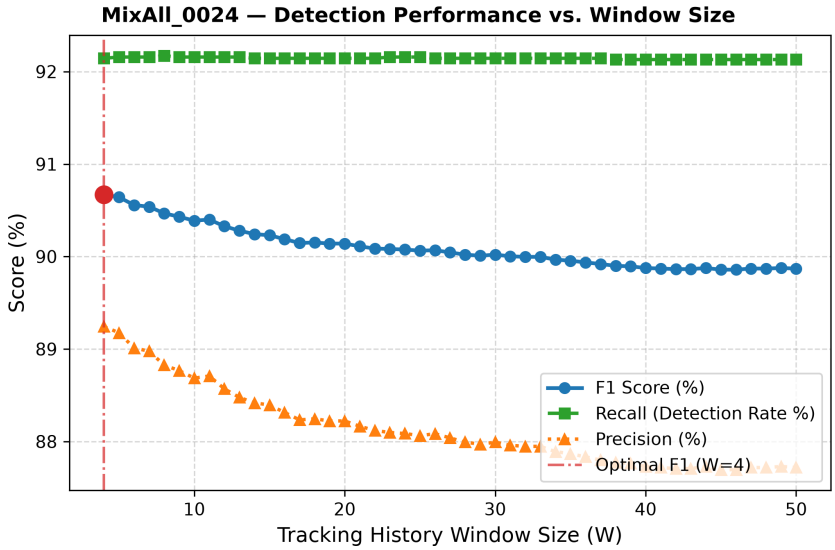


Fig. 5. Service I detection against the history window W on VeReMi Extension MIXALL_0024.

TABLE VI
SERVICE I MEDIAN LATENCY PER MESSAGE ON A STARFIVE VISIONFIVE 2, 50 000 MESSAGES.

Stage	Configuration	Latency (μs)	Relative
Checker	**Service I (OLS-CTRA)**	**1.03**	1
Checker	LegacyChecks	108.65	105.5×
Checker	ExperiChecks	124.11	120.5×
Checker	CaTChChecks [10]	134.10	130.2×
Scorer	**Service I (Beta)**	**0.37**	1
Scorer	ThresholdApp	1.01	2.7×
Scorer	BehavioralApp	1.27	3.4×
Scorer	AggregationApp	3.84	10.4×
End to end	**Service I**	**1.45**	1
End to end	LegacyChecks + ThresholdApp	110.11	75.9×
End to end	LegacyChecks + BehavioralApp	110.53	76.2×
End to end	ExperiChecks + ThresholdApp	125.70	86.7×
End to end	CaTChChecks + BehavioralApp	136.37	94.0×

receivers), `onMotionVectors` takes 170 μs per message on average and about 1 ms at the 99th percentile, 4 965 to 6 252 messages per second on one core. The UKF update takes over half of the time. Association has the longest tail: its p99 is 20 times its mean, because one CPM with many objects reaching a receiver with many tracks makes a large assignment problem. Even at the maximum, 6.1 ms, a message is processed well within the 100 ms minimum CAM interval, although two orders of magnitude above Service I.

REFERENCES

[1] J. Kamel, M. R. Ansari, J. Petit, A. Kaiser, I. B. Jemaa, and P. Urien, "Simulation framework for misbehavior detection in vehicular networks," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 6, pp. 6631–6643, 2020.

TABLE VII
SERVICE II DETECTION ON LUST−FULL, PER RECEIVED VECTOR.

Fault	All vectors		CAM		CPM objects	
	F1	FPR	F1	FPR	F1	FPR
ConstPos	0.8306	6.06%	0.9166	1.84%	0.7861	8.19%
ConstPosOffset	0.2546	6.06%	0.1476	1.84%	0.3001	8.19%
RandomPos	0.8546	6.06%	0.9295	1.84%	0.8163	8.19%
RandomPosOffset	0.8440	6.48%	0.9241	1.99%	0.8030	8.75%
ConstSpeed	0.4845	6.96%	0.6492	1.86%	—	8.72%
ConstSpeedOffset	0.3857	6.49%	0.5188	1.87%	—	8.08%
RandomSpeed	0.3910	6.04%	0.5162	1.87%	—	7.48%
RandomSpeedOffset	0.1996	5.30%	0.2645	1.86%	—	6.48%
All	0.6421	6.18%	0.6650	1.87%	0.6241	7.95%

TABLE VIII
SERVICE II ASSOCIATION ON LUST−FULL, AGAINST THE ORACLE STATION. TRACKS/OBJECT, COMPLETENESS AND ID SWITCHES OVER HONEST VECTORS.

Fault	**Purity**	**Tracks/obj.**	**Compl.**	**ID sw./100**	**CAM rej.**	**CPM rej.**
ConstPos	99.78%	3.46	77.11%	27.95	3.16%	10.94%
ConstPosOffset	99.67%	3.46	77.11%	27.95	3.16%	10.94%
RandomPos	99.77%	3.46	77.11%	27.94	3.16%	10.94%
RandomPosOffset	98.78%	3.48	77.02%	28.03	3.30%	11.18%
ConstSpeed	99.59%	3.93	74.52%	31.75	3.17%	10.09%
ConstSpeedOffset	99.61%	3.87	74.83%	31.64	3.18%	9.71%
RandomSpeed	99.60%	3.93	74.86%	31.75	3.18%	10.11%
RandomSpeedOffset	99.61%	3.89	75.24%	31.68	3.18%	9.65%
All	99.55%	3.69	75.82%	30.09	3.19%	10.34%

TABLE IX
SERVICE II LATENCY ON A STARFIVE VISIONFIVE 2, LUST−FULL, MEAN ± STANDARD DEVIATION OVER THE EIGHT SUBSETS. STAGES 0–2 PER MESSAGE, 3–5 PER VECTOR.

Stage	**Mean (μs)**	**p99 (μs)**	**Share**
`onMotionVectors`	170.19 ± 13.65	1020.91 ± 172.84	100.0%
0 retire	1.61 ± 0.52	33.97 ± 10.35	0.9%
1 plausibility	0.58 ± 0.01	1.25 ± 0.00	0.3%
2 associate	40.50 ± 14.57	806.78 ± 209.81	23.6%
3 checks	23.27 ± 0.27	27.22 ± 3.96	17.0%
4 UKF update	97.62 ± 6.48	415.66 ± 1.89	54.7%
5 score	0.49 ± 0.00	0.75 ± 0.00	0.4%

[2] R. Riebl, H.-J. Günther, C. Facchi, and L. Wolf, "Artery: Extending veins for vanet applications," pp. 450–456, 2015.

[3] L. Codecá, R. Frank, S. Faye, and T. Engel, "Luxembourg SUMO Traffic (LuST) Scenario: Traffic Demand Evaluation," *IEEE Intelligent Transportation Systems Magazine*, vol. 9, no. 2, pp. 52–63, 2017.

[4] J. Kamel, M. Wolf, R. W. van der Hei, A. Kaiser, P. Urien, and F. Kargl, "Veremi extension: A dataset for comparable evaluation of misbehavior detection in vanets," pp. 1–6, 2020.

[5] M. Boban, J. Barros, and O. K. Tonguz, "Geometry-based vehicle-to-vehicle channel modeling for large-scale simulation," *IEEE Transactions on Vehicular Technology*, vol. 63, no. 9, pp. 4146–4164, 2014.

[6] ETSI, "Intelligent Transport Systems (ITS); Vehicular Communications; Basic Set of Applications; Part 2: Specification of Cooperative Awareness Basic Service," European Telecommunications Standards Institute (ETSI), Sophia Antipolis, France, European Standard EN 302 637-2, April 2019. [Online]. Available: https://www.etsi.org/deliver/etsi_en/302600_302699/30263702/

[7] ——, "Intelligent Transport Systems (ITS); Vehicular Communications; Basic Set of Applications; Collective Perception Service; Release 2," European Telecommunications Standards Institute (ETSI), Sophia Antipolis, France, Technical Specification TS 103 324, June 2023. [Online]. Available: https://standards.iteh.ai/catalog/standards/etsi/14dac0c0-92d4-40b9-956c-11b83e9cef11/etsi-ts-103-324-v2-1-1-2023-06

[8] ——, "Intelligent transport systems (its); vehicular communications; basic set of applications; analysis of the collective perception service (cps); release 2," European Telecommunications Standards Institute (ETSI), Sophia Antipolis, France, Technical Report TR 103 562, December 2019.

[9] R. Sultana, J. Grover, M. Tripathi, and P. Sharma, "La-detects: Local and adaptive data-centric misbehavior detection framework for vehicular technology security," *IEEE Open Journal of Vehicular Technology*, vol. 6, pp. 145–169, 2025.

[10] J. Kamel, A. Kaiser, I. ben Jemaa, P. Cincilla, and P. Urien, "Catch: A confidence range tolerant misbehavior detection approach," *2019 IEEE Wireless Communications and Networking Conference (WCNC)*, pp. 1–8, 2019.